

Virtual Address Translation and Page Tables

1. Función `translate_address`:

Propósito: Es el núcleo del sistema (MMU - Memory Management Unit).

Traduce una dirección virtual de 16 bits a una dirección física.

- **¿Qué hace?**

1. **Validación de Rango:** Verifica que la dirección no exceda el límite de 16 bits (\$0xFFFF\$).
2. **Extracción de VPN y Offset:** Usa operaciones de bits para separar la dirección:
 - offset: Los últimos 8 bits (& 0xFF).
 - vpn (Virtual Page Number): Los 8 bits superiores (>> 8).
3. **Verificación de Tabla:** Revisa si la página virtual está marcada como valid en la tabla de páginas del proceso.
4. **Cálculo Físico:** Si es válida, multiplica el número de marco físico (PFN) por el tamaño de página (256 bytes) y suma el desplazamiento (offset) para obtener la **Dirección Física (PA)**.

2. Función `init_ram`:

Propósito: Inicializa el estado de la memoria RAM física antes de cargar cualquier proceso.

- **¿Qué hace?**

1. Limpia toda la RAM marcando los marcos como FREE (0).
2. **Simulación de Sistema Operativo:** Ocupa aleatoriamente entre 10 y 60 marcos con la etiqueta SYSTEM. Esto simula que el kernel ya está usando parte de la memoria.
3. **Garantía de Espacio:** Utiliza un bucle do-while para asegurarse de que, tras la fragmentación aleatoria, quede suficiente espacio libre para los procesos que se van a cargar.

3. Función `print_ram_map`:

Propósito: Genera una representación visual en la terminal del estado actual de la RAM.

- **¿Qué hace?**

1. Imprime una cuadrícula de 10x10 que representa los 100 marcos (NUM_FRAMES).
2. Utiliza **colores ANSI** para diferenciar quién es dueño de cada marco:

- **Verde (F):** Libre.
 - **Rojo (X):** Sistema Operativo.
 - **Azul (1):** Proceso 1.
 - **Magenta (2):** Proceso 2.
3. Al final, muestra un resumen estadístico del conteo de marcos por cada dueño.

4. Función `allocate_frame`:

Propósito: Buscar y reservar un espacio físico disponible en la RAM.

- **¿Qué hace?**
 1. Recorre el arreglo `ram_fisica` de forma secuencial.
 2. Al encontrar el primer marco con estado `FREE`, lo cambia al ID del proceso que lo solicitó (`owner_id`).
 3. Retorna el índice del marco encontrado o -1 si la memoria está llena.

5. Función `load_process`:

Propósito: "Cargar" un proceso en memoria, vinculando sus páginas virtuales con marcos físicos.

- **¿Qué hace?**
 1. **Sanity Check:** Primero verifica si hay suficientes marcos libres en total antes de intentar asignar nada.
 2. **Asignación Gradual:** Intenta asignar un marco para cada página requerida por el proceso.
 3. **Mecanismo de Rollback (Crítico):** Si durante la carga se queda sin memoria (aunque el check inicial haya pasado), el código "deshace" lo hecho: libera los marcos ya asignados y limpia la tabla de páginas para evitar dejar basura en la RAM.
 4. **Actualización de Tabla:** Si tiene éxito, marca las entradas de la `PageTableEntry` como válidas y guarda el PFN correspondiente.

6. Función `main`:

Propósito: Orquestrar la ejecución completa del simulador.

- **¿Qué hace?**
 1. **Parámetros de entrada:** Recibe por consola el número de páginas, el nombre de un archivo con direcciones y una semilla opcional para el azar.
 2. **Preparación:** Inicializa las tablas de páginas en 0 e invoca a `init_ram`.

3. **Carga de Procesos:** Intenta cargar dos procesos (P1 y P2) con la misma cantidad de páginas.
4. **Procesamiento Batch:** Abre el archivo de texto proporcionado, lee cada dirección virtual y ejecuta `translate_address` para ambos procesos, permitiendo comparar cómo una misma dirección virtual mapea a distintos puntos físicos en cada proceso.

```
aldo-reyes@aldo-reyes-QEMU-Virtual-Machine:~/Escritorio/Labs/lab8$ make run
--- MAPA INICIAL ---
--- VISUALIZACIÓN DE RAM (10x10) ---
 0:F  1:F  2:F  3:F  4:X  5:F  6:F  7:F  8:F  9:F
10:X 11:X 12:X 13:X 14:X 15:X 16:F 17:F 18:F 19:X
20:X 21:X 22:X 23:X 24:X 25:X 26:X 27:F 28:X 29:F
30:F 31:F 32:F 33:F 34:F 35:F 36:X 37:F 38:F 39:X
40:F 41:F 42:X 43:X 44:F 45:F 46:X 47:X 48:F 49:F
50:X 51:X 52:F 53:F 54:F 55:X 56:F 57:F 58:F 59:X
60:F 61:X 62:X 63:F 64:F 65:F 66:F 67:F 68:X 69:F
70:F 71:X 72:F 73:X 74:F 75:X 76:X 77:F 78:X 79:F
80:F 81:X 82:F 83:F 84:F 85:F 86:X 87:F 88:F 89:X
90:X 91:F 92:F 93:F 94:X 95:F 96:X 97:X 98:F 99:X

ESTADO: LIBRE:58 SISTEMA:42 P1:0 P2:0
Load process P2: V=20 -> VPN 0..19 mapped to PFNs [0, 1, 2, 3, 5, 6, 7, 8, 9, 16, 17, 18, 27, 29, 30,
31, 32, 33, 34, 35]

Proceso 1 cargado. Load process P3: V=20 -> VPN 0..19 mapped to PFNs [37, 38, 40, 41, 44, 45, 48, 49, 5
2, 53, 54, 56, 57, 58, 60, 63, 64, 65, 66, 67]

Proceso 2 cargado.
```

--- MAPA DESPUÉS DE CARGAR PROCESOS ---

--- VISUALIZACIÓN DE RAM (10x10) ---

0:1	1:1	2:1	3:1	4:X	5:1	6:1	7:1	8:1	9:1
10:X	11:X	12:X	13:X	14:X	15:X	16:1	17:1	18:1	19:X
20:X	21:X	22:X	23:X	24:X	25:X	26:X	27:1	28:X	29:1
30:1	31:1	32:1	33:1	34:1	35:1	36:X	37:2	38:2	39:X
40:2	41:2	42:X	43:X	44:2	45:2	46:X	47:X	48:2	49:2
50:X	51:X	52:2	53:2	54:2	55:X	56:2	57:2	58:2	59:X
60:2	61:X	62:X	63:2	64:2	65:2	66:2	67:2	68:X	69:F
70:F	71:X	72:F	73:X	74:F	75:X	76:X	77:F	78:X	79:F
80:F	81:X	82:F	83:F	84:F	85:F	86:X	87:F	88:F	89:X
90:X	91:F	92:F	93:F	94:X	95:F	96:X	97:X	98:F	99:X

ESTADO: LIBRE:18 SISTEMA:42 P1:20 P2:20

--- TRADUCCIONES BATCH ---

P1: VA=0x0000 (0) VPN=0x00 OFF=0x00 PFN=0 PA=0x0000

P2: VA=0x0000 (0) VPN=0x00 OFF=0x00 PFN=37 PA=0x2500

P1: VA=0x00FF (255) VPN=0x00 OFF=0xFF PFN=0 PA=0x00FF

P2: VA=0x00FF (255) VPN=0x00 OFF=0xFF PFN=37 PA=0x25FF

P1: VA=0x0100 (256) VPN=0x01 OFF=0x00 PFN=1 PA=0x0100

P2: VA=0x0100 (256) VPN=0x01 OFF=0x00 PFN=38 PA=0x2600

P1: VA=0x0200 (512) VPN=0x02 OFF=0x00 PFN=2 PA=0x0200

P2: VA=0x0200 (512) VPN=0x02 OFF=0x00 PFN=40 PA=0x2800

P1: VA=0x0F00 (3840) VPN=0x0F OFF=0x00 PFN=31 PA=0x1F00

P2: VA=0x0F00 (3840) VPN=0x0F OFF=0x00 PFN=63 PA=0x3F00

P1: VA=70000 ERROR=VA_OUT_OF_RANGE

P2: VA=70000 ERROR=VA_OUT_OF_RANGE